

Strings

What is a string?

- In many other programming languages, string is a sequence of characters.
- In Java, strings are objects.
- Strings are surrounded by double quotes.
- Strings are immutable. Once a string is created, it can't be changed.
- If we want to to modify a string, it will create a new string which contains the modifications.

String Input and Output

- `.next()` and `.nextLine()` of the scanner class is used to take string as input.
- `.next()` only takes input until it finds a “ ” or space in the given input.
- `.nextLine()` takes input until it finds a “\n” in the given input.

```
1. class stringInput{
2.     public static void main(String[] args) {
3.         Scanner sc = new Scanner(System.in);
4.         String input1 = sc.next();
5.         String input2 = sc.nextLine();
6.         System.out.println(input1);
7.         System.out.println(input2);
8.     }
```

Welcome to Dr.Java Working directory is C:\Users\Desktop

> CSE IS FUN.

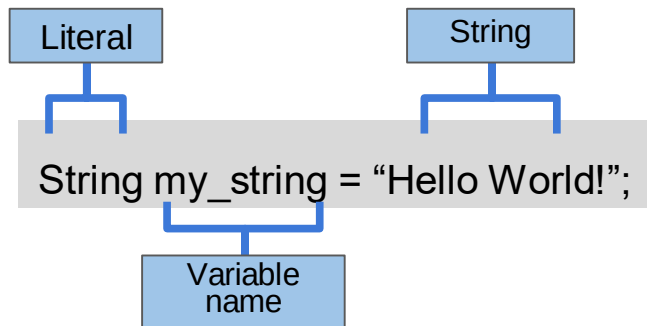
> CSE IS FUN.

CSE

CSE IS FUN.

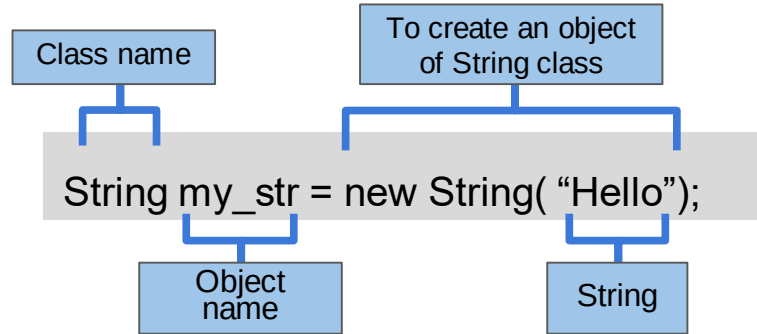
String Initialization

- Initialization using String literals:



String Initialization

- Creating an object of String class:



String Length

- The number of characters in a string.
- Can find using the length() method.

```
1. class stringLength {  
2.     public static void main(String[] args) {  
3.         String str1 = "Hello World";  
4.         String str2 = ""; // Empty string  
5.         System.out.println(str1.length());  
6.         System.out.println(str2.length());  
7.     }  
8. }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop  
> run stringLength  
11  
0
```

String Indexing

- String indexing starts from 0 and ends at (Length of string - 1).
- If we create `string1 = "CSE IS FUN."`, which has a length of 11, the indexing will look like this:

0	1	2	3	4	5	6	7	8	9	10
C	S	E		I	S		F	U	N	.

- Blanks and special characters are also counted as characters.
- We can get characters at each index by using `charAt(index)` method.
Example: `string1.charAt(2)` will return "E".

Iterating a String

- Accessing the characters of a string one by one.
- Iterating a string using for loop and charAt() method.

```
1. class StringIteration {  
2.     public static void main(String[] args) {  
3.         String str = new String("Hello");  
4.         for (int i = 0; i < str.length(); i++){  
5.             System.out.println(str.charAt(i));  
6.         }  
7.     }  
8. }
```

Welcome to DrJava. Working directory is C:\Users\Desktop

> run StringIteration

H

e

l

l

o

String Concatenation

- “+” is used to concatenate two or more strings together.
- New string created after concatenation.
- If a different data type follows a string data type in concatenation, that data is automatically converted to string.

```
1. String str1 = "We are learning";
2. String str2 = "Java.";
3. int num = 123;
4. String str3 = str1 + str2;
5. String str4 = str1 + num;
6. System.out.println(str3);
7. System.out.println(str4);
```

```
Welcome to DrJava. Working directory
is C:\Users\Desktop
We are learning Java.
We are learning 123
```

Comparing Strings

- `String str1 = new String("Hello"); String str2 = new String("hello");`

Methods	Functions	Method Call	Value returned
<code>.equals()</code>	Checks if values in two strings are same. Case sensitive.	<code>str1.equals(str2)</code>	False
<code>.equalsIgnoreCase()</code>	Checks if values in two strings are same. Not case sensitive.	<code>str1.equalsIgnoreCase(str2)</code>	True
<code>==</code> or <code>equals</code> operator	Checks if reference (address) of two strings are same.	<code>str1 == str2</code>	False
<code>.compareTo()</code>	Compares two string values lexicographically. Returns 0 when identical, (+)ve when first string is greater and (-)ve values when second string is greater respectively.	<code>str1.compareTo(str2)</code>	-32 (The output is -32 since "H" is 32 less than "h" lexicographically)

Case Conversion

- **.toLowerCase()**
 - Converts all characters to lowercase letters.
- **.toUpperCase()**
 - Converts all characters to uppercase letters.

```
1. String str1 = "LearNinG IS Fun";
2. String str1_lower= str1.toLowerCase()
3. String str1_upper = str1.toUpperCase()
4. System.out.println(str1_lower)
5. System.out.println(str1_upper)
```

```
Welcome to DrJava. Working directory is
C:\Users\Desktop
>learning is fun
LEARNING IS FUN
```

.charAt(index) Method

- **.charAt(index)**
 - Returns the character at index value passed as argument.

```
1. String str1 = "Hello World";  
2. System.out.println(str1.charAt(3))
```

```
Welcome to DrJava. Working directory is  
C:\Users\Desktop  
>○
```

ASCII values

- Unique Unicode value assigned to all keyboard characters.
- Range of ASCII values of uppercase characters ('A' to 'Z'): 65 - 90
- Range of ASCII values of lowercase characters ('a' to 'z'): 97 - 122
- ASCII value of space: 32

See more from:

<https://www.cs.cmu.edu/~pattis/15-1XX/common/handouts/ascii.html>

Converting a Character to ASCII Value

- `.codePointAt(index)`
 - Takes an index of a string as a parameter and returns the ASCII value of the character of that index.
- `(int) chr`
 - Can simply cast a character as an integer.

```
1. String str1 = "CSE110";
2. int unicode_str1 = str1.codePointAt(0);
3. System.out.println(unicode_str1)
```

```
Welcome to DrJava. Working directory is
C:\Users\Desktop
>67
```

```
1. char c1 = 'a';
2. int unicode_c1 = (int) c1;
3. System.out.println(unicode_c1)
```

```
Welcome to DrJava. Working directory is
C:\Users\Desktop
>97
```

Converting an ASCII value to its Character

- **(char) int**
 - Converts an ASCII value to its character version.

```
1. int ascii1 = 65;  
2. char character = (char) ascii1;  
3. System.out.println(character );
```

```
Welcome to DrJava. Working directory is  
C:\Users\Desktop  
>A
```

Can you now try to figure out how you can convert an uppercase character to its equivalent lowercase character using the knowledge you gained on ASCII values so far?

THANK YOU!